Diagrama de Classe

CSU04: Enviar mensagem

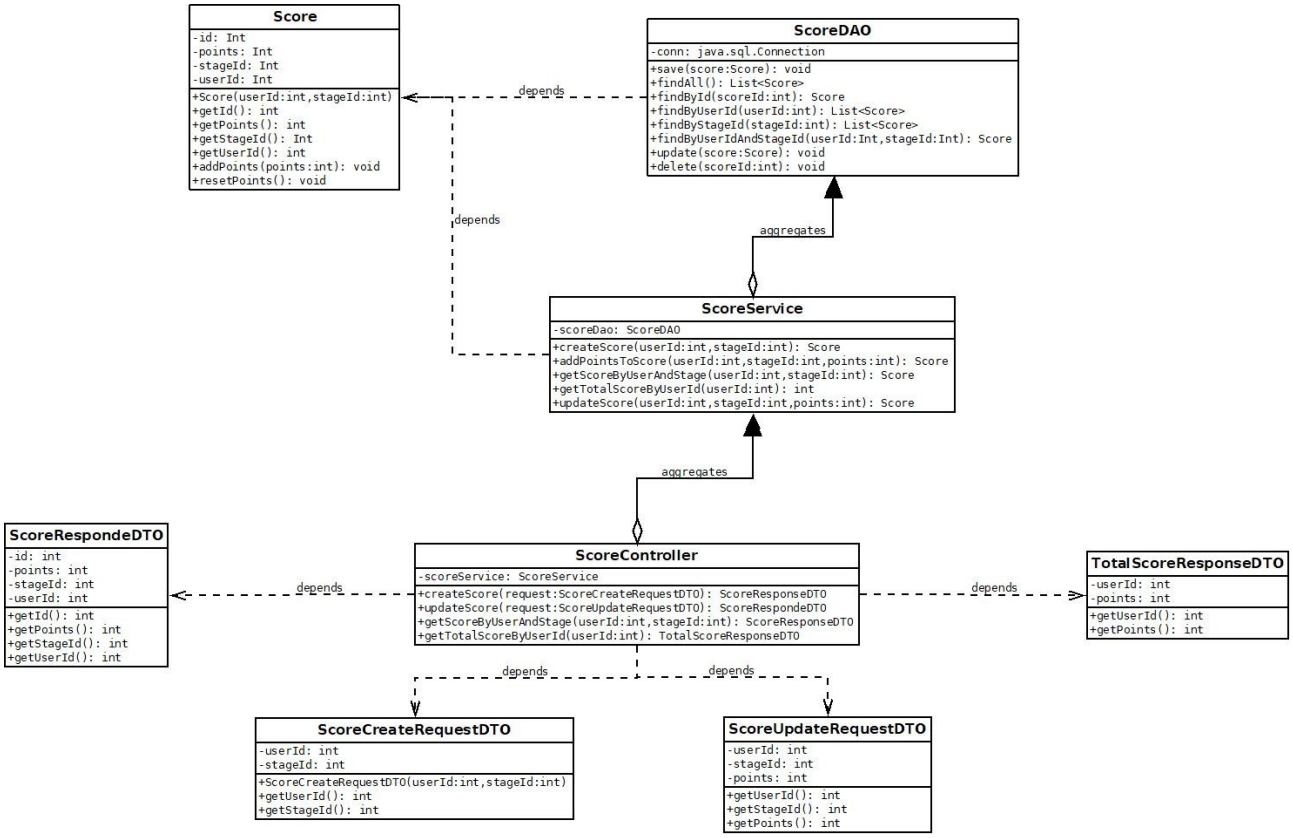
CSU06: Fase por dentro da fibra óptica

Autor: Ana Carolina Fernandes (Arquiteta de Software)

Descrição do caso de uso:

Os casos de uso apresentados permitem ao sistema gerenciar a pontuação obtida pelo aluno ao completar as fases do jogo. O sistema deve ser capaz de salvar a pontuação de um jogador para uma fase específica e, posteriormente, recuperar essas informações.

Diagrama:



Descrição do diagrama: Seguindo a arquitetura em camadas do projeto, o sistema de pontuação é dividido em componentes:

- **Entidade principal:**
    - A classe `Score` representa a entidade de pontuação, contendo informações como identificador (`id`), quantidade de pontos (`points`), o identificador da fase (`stageId`) e do usuário (`userId`).
- **Persistência:**

    - A classe `ScoreDAO` é a única responsável por todas as operações de banco de dados relacionadas à pontuação (salvar, buscar, atualizar e deletar).
- **Serviços:**
    - A classe `ScoreService` encapsula toda a lógica de negócio referente à pontuação. Ela utiliza uma instância de `ScoreDAO` para criar `(createScore)` e/ou adicionar novos pontos a uma pontuação existente `(addPointsToScore)`.
- **Controladores:**
    - A classe `ScoreController` recebe as chamadas do cliente, utiliza o DTOs para extrair os dados e os repassa para o `ScoreService`, que processa a requisição.
- **Objetos de Transferência de Dados (DTOs):**
    - O sistema utiliza um padrão que separa os DTOs de entrada (Request) e saída (Response).
    - As classes `ScoreCreateRequestDTO` encapsula os dados necessários para criar uma pontuação em uma determinada fase (`userId e stageId`).
    - A classe `ScoreUpdateRequestDTO` encapsula os dados necessários para atualizar uma pontuação já existente com uma determinada quantidade de pontos (`points`).
    - A classe `ScoreResponseDTO` formata os dados de resposta que serão enviados para o cliente.

Implementação: Para implementar o sistema, recomenda-se a seguinte abordagem em camadas:

1. **Modelagem das entidades:** Crie a classe `Score` para representar os dados centrais.
2. **Camada de persistência:** Implemente `ScoreDAO` para interagir com o banco de dados.
3. **Regras de negócio:** Crie `ScoreService`, que conterá a lógica de negócio, utilize a instância de `ScoreDAO` para realizar validações, como buscar uma pontuação existente antes de adicionar novos pontos a ela.

4. **Camada de controle:** Construa a `ScoreController`, que receberá as requisições HTTP. Seus métodos usarão os `RequestDTOs`, para receber os dados, chamar o serviço correspondente e retornar o `ResponseDTO` adequado.
5. **Transferência de dados:** Crie as classes `ScoreCreateRequestDTO`, `ScoreUpdateRequestDTO`, `ScoreResponseDTO` e `TotalScoreResponseDTO` e as utilize na camada de controle para receber dados (`Request`),e para expor apenas as informações necessárias como resposta (`Response`), evitando expor detalhes internos das entidades.